

Министерство науки и высшего образования Российской Федерации
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Институт прикладной математики и компьютерных наук

Отчет по лабораторной работе №2
по дисциплине «Разработка средств защиты информации»

Выполнил:
студент группы № 932224
Ликонцев Н.Н

Проверил:
Преподаватель
Андреева В.В.

Томск – 2026

1. Введение

В рамках лабораторной работы проведён комплексный анализ безопасности SAT-решателя -приложения на языке C++14, реализующего алгоритм DPLL с пользовательским аллокатором памяти (endurodave/Allocator). Анализ выполнен двумя независимыми инструментами:

- Статический анализ -PVS-Studio (анализ исходного кода без запуска программы).
- Динамический анализ -Valgrind Memcheck (обнаружение ошибок во время выполнения).

1.1. Объект анализа

Исполняемый файл SAT_Solver, собранный из следующих компонентов:

Компонент	Описание
main.cpp	Точка входа, DPLL-решатель (функция solveDPLL)
BBV.cpp/.h	Битовый вектор -основная структура данных
boolinterval.cpp/.h	Булев интервал (вектор значений + don't-care маска)
boolequation.cpp/.h	Модель SAT-задачи; правила упрощения DPLL
*branching_strategy	Стратегии выбора переменной ветвления (паттерн «Стратегия»)
benchmarks_example	Инфраструктура бенчмарков аллокатора
Allocator-1.0.0	Внешняя библиотека пула памяти

1.2. Входные данные

Для Valgrind Memcheck в качестве тестового входа использован файл Sat_ex30_3.pla (30 переменных, 40 клозов):

```
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all \  
--track-origins=yes --xml=yes --xml-file=valgrind_report.xml \  
./SAT_Solver ../SatExamples/Sat_ex30_3.pla
```

Для PVS-Studio анализировался весь исходный код проекта:

```
pvs-studio-analyzer analyze -o pvs_report.log -j4  
plog-converter -a GA:1,2 -t json -o reports/pvs_studio/pvs_report pvs_report.log
```

2. Статический анализ -PVS-Studio

2.1. Общая статистика

Источник	Предупреждений	% от общего
Исходники проекта	522	86%
Библиотека Allocator	83	14%
Итого	605	100%

Уровень	Количество (проект)	Описание
Уровень 1	352	Высокий -потенциально критичные ошибки
Уровень 2	140	Средний -подозрительный код
Уровень 3	30	Низкий -рекомендации по стилю

2.2. Ключевые находки (критичные и важные)

Ниже перечислены предупреждения, влияющие на корректность, безопасность или надёжность программы.

Код	Уровень	Кол-во	Описание	Файл / строка
V755	Ур. 2	1	Потенциальное переполнение буфера при копировании данных из std::cin	BBV.cpp:437
V690	Ур. 2	4	Нарушение правила трёх: есть operator=, но нет конструктора копирования	BBV.h:9 NodeBoolTree.h:7 boolequation.h:9 +1 ещё
V1077	Ур. 2	4	Поля ptr, index могут остаться неинициализированными в конструкторе X	BBV.cpp:22 BBV.cpp:25 BBV.cpp:27 +12 ещё
V668	Ур. 2	8	Проверка указателя на NULL после new бессмысленна -исключение возникает раньше	BBV.cpp:26 BBV.cpp:43 BBV.cpp:73 +5 ещё
V2565	Ур. 2	1	Косвенная рекурсия solveDPLL (строки 35 и 58)	main.cpp:35 main.cpp:58
V108	Ур. 2	67	Индекс массива cnf[] имеет тип int вместо size_t	benchmarks_example.h:42 benchmarks_example.h:49 BBV.cpp:30 +64 ещё
V302	Ур. 2	1	BBV::operator[] принимает int вместо size_t	BBV.h:47
V688	Ур. 3	2	Аргумент функции size совпадает по имени с полем класса	BBV.h:54 boolequation.cpp:11
V818	Ур. 2	2	Инициализация через operator= в теле конструктора; предпочтителен список инициализации	boolequation.cpp:11 boolequation.cpp:22 boolinterval.cpp:47 +2 ещё
V820	Ур. 3	2	Переменная не используется после копирования; копирование избыточно	boolequation.cpp:22 boolinterval.cpp:106

2.3. Детальный разбор критичных предупреждений

V755 -Возможное переполнение буфера (BBV.cpp:437)

Метод BBV::operator>>(std::istream&) читает строку из cin в статический символьный буфер фиксированного размера без проверки длины вводимых данных. Злоумышленник, имея возможность управлять стандартным вводом программы, может переполнить буфер и перезаписать смежные области стека.

Рекомендация: заменить char[] на std::string, использовать std::getline.

V690 -Нарушение правила трёх (BBV.h, boolinterval.h, boolequation.h, NodeBoolTree.h)

Классы X, BoolInterval, BoolEquation и NodeBoolTree имеют operator= и деструктор с ручным управлением памятью, но не объявляют конструктор копирования. При копировании объекта компилятор генерирует поверхностное (shallow) копирование указателей, что при последующем уничтожении копии и оригинала ведёт к двойному освобождению одного и того же блока памяти (double-free) -неопределённому поведению.

Рекомендация: добавить конструктор копирования с глубоким копированием либо запретить копирование (= delete).

V1077 -Неинициализированные поля конструктора X (BBV.cpp)

Конструктор класса X использует условную инициализацию полей ptr и index: при определённых путях выполнения поля остаются неинициализированными. Обращение к такому объекту приводит к неопределённому поведению.

Рекомендация: явно инициализировать все поля в списке инициализации конструктора.

V668 -Бессмысленная проверка указателя после new (BBV.cpp)

В восьми местах кода после operator new() выполняется проверка вида if (vec != nullptr). Согласно стандарту C++, operator new() никогда не возвращает nullptr -при нехватке памяти бросается std::bad_alloc. Проверка создаёт ложное ощущение защищённости, скрывая неустановленный обработчик исключения.

Рекомендация: удалить проверки, установить std::set_new_handler для корректной обработки исчерпания памяти.

2.4. Стиливые и MISRA-предупреждения

Перечисленные предупреждения не влияют на текущую корректность работы программы, но указывают на отступления от стандарта MISRA C++ и общепринятых практик безопасного кодирования.

Код(ы)	Кол-во	Описание
V2507 / V3504	130	Тело if/else не заключено в фигурные скобки
V2506	70	Функция имеет несколько точек выхода (return)
V2563 / V3539	72	Арифметика указателей вместо индексирования массива
V2575 / V3549	60	Объявления в глобальном пространстве имён
V2511	26	Использование raw new (MISRA C++: предпочтительны умные указатели)
V2513 / V3508	5	Использование strlen (MISRA: небезопасная функция)
V2547 / V3527	6	Возвращаемое значение set_new_handler не используется
V2578 / V3551	16	Строковый литерал неявно распадается в char*

3. Динамический анализ -Valgrind Memcheck

3.1. Общая статистика

Файл входных данных: Sat_ex30_3.pla (30 переменных, 40 клозов)

Категория	Записей	Утечка (байт)
Definitely Lost	69	884,066
Indirectly Lost	16	920
Итого	85	884,986

Ошибок обращения к неинициализированной памяти (Invalid read/write) и выхода за границы массива обнаружено не было.

3.2. Причины утечек

Все 85 записей о потере памяти связаны с одной архитектурной проблемой: класс BoolEquation выделяет массив BoolInterval** cnf в конструкторе, но не имеет деструктора -массив никогда не освобождается.

Механизм утечки:

1. Начальный объект BoolEquation создаётся в main() через new -cnf выделяется в куче.
2. При каждом шаге ветвления DPLL solveDPLL() создаёт копию BoolEquation на стеке.
3. Копирующий конструктор выделяет второй массив cnf (boolequation.cpp:28).
4. При выходе из solveDPLL() копия уничтожается, но деструктор не освобождает cnf.
5. BBV-буферы внутри BoolInterval теряются косвенно через утерянный указатель cnf.

3.3. Группы утечек по стеку вызовов

Тип	Кол-во	Байт	Стек вызовов
Definitely Lost	46	799,848	BBV::operator char*() → BoolEquation::CheckRules() → solveDPLL(BoolEquation*, BranchingStrategy const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >*)
Definitely Lost	2	51,520	BBV::operator char*() → BoolInterval::BoolInterval(char const*) → main
Definitely Lost	9	28,160	BoolEquation::BoolEquation(BoolEquation&) → solveDPLL(BoolEquation*, BranchingStrategy const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >*) → solveDPLL(BoolEquation*, BranchingStrategy const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >*)
Definitely Lost	8	1,656	Allocator::Allocate(unsigned long) → BoolInterval::operator new(unsigned long) → BoolEquation::BoolEquation(BoolEquation&)
Definitely Lost	1	1,280	BoolEquation::BoolEquation(BoolInterval**, BoolInterval*, int, int, BBV) → main
Definitely Lost	1	1,280	BoolEquation::BoolEquation(BoolEquation&) → solveDPLL(BoolEquation*, BranchingStrategy const&, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >*) → main
Definitely Lost	2	322	BBV::operator char*() → BoolInterval::BoolInterval(unsigned long) → main
Indirectly Lost	16	920	BBV::operator=(BBV&) → BoolInterval::BoolInterval(BBV&, BBV&) → BoolEquation::BoolEquation(BoolEquation&)

3.4. Дополнительные утечки из BBV::operator char*()

Метод BBV::operator char*() (BBV.cpp:309) выделяет новый char-буфер через new[] и возвращает его вызывающей стороне. В BoolEquation::CheckRules() этот буфер используется как временный для сравнения строк, но ни один из вызывающих не освобождает его через delete[]. Это дополнительный источник утечек, выявленный в 53 из 69 записей категории Definitely Lost (суммарно ~878 000 байт).

```
// BBV.cpp:309 -каждый вызов выделяет новый буфер
BBV::operator char*() {
    char* str = new char[len + 1];
    // ... заполнение ...
    return str;    // вызывающая сторона обязана вызвать delete[]
}
```

3.5. Рекомендации по устранению

Проблема	Файл	Рекомендуемое исправление
Нет деструктора BoolEquation	boolequation.cpp/h	Добавить ~BoolEquation() { delete[] cnf; }
BBV::operator char*() возвращает heap-буфер	BBV.cpp:309	Вернуть std::string или задокументировать, что вызывающая сторона обязана вызвать delete[]
Копирующий конструктор BoolEquation не освобождает cnf	boolequation.cpp:28	Убедиться, что старый cnf освобождается перед re-allocation в operator=

Исправление деструктора (минимальный патч):

```
// boolequation.h
class BoolEquation {
    // ... existing members ...
    ~BoolEquation();
};

// boolequation.cpp
BoolEquation::~BoolEquation() {
    delete[] cnf;
}
```

4. Сводная таблица уязвимостей

В таблице ниже собраны все выявленные проблемы с указанием инструмента, критичности и рекомендуемого действия.

Инструмент	ID / Тип	Критичность	Краткое описание	Действие
Valgrind	Definitely Lost	Высокая	69 утечек, 884,066 байт -отсутствует ~BoolEquation()	Добавить деструктор delete[] cnf
Valgrind	Indirectly Lost	Средняя	16 утечек, 920 байт -буферы BBV через утерянный cnf	Устранится вместе с ~BoolEquation()
Valgrind	BBV::op char*()	Средняя	Возвращает new[] буфер без delete[] у вызывающих; 53 из 69 записей Definitely Lost	Вернуть std::string или задокументировать контракт
PVS V755	CWE-120	Высокая	Переполнение буфера: std::cin → char[] без проверки длины (BBV.cpp:437)	Заменить на std::string + getline
PVS V690	Ур. 2 × 4	Высокая	Нарушение правила трёх в X, BoolInterval, BoolEquation, NodeBoolTree -риск double-free	Добавить copy ctor или запретить копирование
PVS V1077	Ур. 2 × 15	Средняя	Поля ptr, index могут не инициализироваться в конструкторе X	Инициализировать в списке инициализации
PVS V668	Ур. 2 × 8	Средняя	Бессмысленные null-проверки после new в BBV.cpp	Удалить проверки, добавить set new handler
PVS V2565	Ур. 2 × 2	Информац.	Рекурсивная функция solveDPLL -MISRA-несоответствие	Допустимо, задокументировать
PVS V108	Ур. 2 × 67	Низкая	Индекс массива cnf[] имеет тип int вместо size_t -потенциальный переполнение на 64-бит	Заменить int на size_t / ptrdiff_t
PVS V302	Ур. 2 × 1	Низкая	BBV::operator[] принимает int вместо size_t	Изменить тип аргумента
PVS style	MISRA ×300+	Низкая	Отсутствие скобок, множественные return, отсутствие фигурных скобок в else и пр.	Рефакторинг при следующей итерации

5. Заключение

Проведённый анализ выявил ряд уязвимостей и дефектов качества в исходном коде SAT-решателя. Ключевые выводы:

1. Наиболее критичная проблема -отсутствие деструктора в классе BoolEquation -приводит к утечке ~885 КБ памяти при решении задачи Sat_ex30_3.pla. Патч требует добавления одной строки (~BoolEquation() { delete[] cnf; }).
2. Потенциальная уязвимость безопасности (V755) -переполнение буфера из стандартного ввода -может быть эксплуатирована при использовании программы в интерактивном режиме.
3. Нарушение правила трёх (V690) в четырёх классах создаёт риск двойного освобождения памяти при копировании объектов.
4. Внешняя библиотека Allocator содержит 83 предупреждения, преимущественно о размерах типов (V122). Исправление в рамках данного проекта нецелесообразно.
5. Большинство предупреждений уровня 1 (стиль, MISRA) не несут прямой угрозы корректности, но снижают читаемость и сопровождаемость кода.

Инструменты использованы: Valgrind Memcheck 3.x, PVS-Studio (Community edition).